

IntelliInsight — Professional Workspace-Based Frontend Refactor

This refactor completely changes your frontend architecture from:

File Explorer Style

into:

Professional AI Workspace Style

Final UX Flow

1. User opens app
2. Uploads multiple files OR connects sources
3. Processing screen appears
4. Files are embedded/stored
5. AI Workspace automatically opens
6. User chats instantly

New Folder Structure

```
src/
├── App.jsx
├──
├── pages/
│   ├── UploadWorkspace.jsx
│   └── AIWorkspace.jsx
├──
├── components/
│   ├── UploadDropzone.jsx
│   ├── ProcessingScreen.jsx
│   ├── ChatInterface.jsx
│   ├── SourceChips.jsx
│   ├── GoogleSheetModal.jsx
│   └── DatabaseModal.jsx
```

```
└─ services/  
  └─ api.js
```

1. api.js

```
import axios from "axios";

const API = axios.create({
  baseURL: "http://localhost:8000",
});

export default API;

// MULTI FILE UPLOAD
export const createWorkspace = async (files) => {
  const form = new FormData();

  files.forEach((file) => {
    form.append("files", file);
  });

  return API.post("/workspace/upload", form, {
    headers: {
      "Content-Type": "multipart/form-data",
    },
  });
};

// CHAT
export const sendWorkspaceQuery = (payload) => {
  return API.post("/workspace/query", payload);
};

// GOOGLE SHEETS
export const connectGoogleSheet = (payload) => {
  return API.post("/workspace/google-sheet", payload);
};

// DATABASE
export const connectDatabase = (payload) => {
  return API.post("/workspace/database", payload);
};
```

2. App.jsx

```
import { useState } from "react";
import UploadWorkspace from "../pages/UploadWorkspace";
import AIWorkspace from "../pages/AIWorkspace";

export default function App() {
  const [workspace, setWorkspace] = useState(null);

  return (
    <div className="app-root">
      {!workspace ? (
        <UploadWorkspace
          onWorkspaceReady={(data) => {
            setWorkspace(data);
          }}
        />
      ) : (
        <AIWorkspace workspace={workspace} />
      )}
    </div>
  );
}
```

3. UploadWorkspace.jsx

```
import { useState } from "react";
import UploadDropzone from "../components/UploadDropzone";
import ProcessingScreen from "../components/ProcessingScreen";
import GoogleSheetModal from "../components/GoogleSheetModal";
import DatabaseModal from "../components/DatabaseModal";
import {
  createWorkspace,
  connectGoogleSheet,
  connectDatabase,
} from "../services/api";

export default function UploadWorkspace({ onWorkspaceReady }) {
  const [processing, setProcessing] = useState(false);
  const [showSheet, setShowSheet] = useState(false);
  const [showDatabase, setShowDatabase] = useState(false);

  const handleFiles = async (files) => {
    try {
      setProcessing(true);
    }
  }
}
```

```

    const res = await createWorkspace(files);

    onWorkspaceReady(res.data);
  } catch (err) {
    console.error(err);
    alert("Upload failed");
  } finally {
    setProcessing(false);
  }
};

const handleGoogleSheet = async (payload) => {
  try {
    setProcessing(true);

    const res = await connectGoogleSheet(payload);

    onWorkspaceReady(res.data);
  } catch (err) {
    console.error(err);
    alert("Google Sheet connection failed");
  } finally {
    setProcessing(false);
  }
};

const handleDatabase = async (payload) => {
  try {
    setProcessing(true);

    const res = await connectDatabase(payload);

    onWorkspaceReady(res.data);
  } catch (err) {
    console.error(err);
    alert("Database connection failed");
  } finally {
    setProcessing(false);
  }
};

return (
  <div className="upload-page">
    {processing && <ProcessingScreen />}

    <div className="upload-center">
      <div className="hero-badge">AI Financial Intelligence</div>

      <h1>IntelliInsight</h1>

      <p>

```

```

    Upload structured and unstructured data sources to start AI-powered
    analysis instantly.
  </p>

  <UploadDropzone onUpload={handleFiles} />

  <div className="connect-buttons">
    <button onClick={() => setShowSheet(true)}>
      Connect Google Sheets
    </button>

    <button onClick={() => setShowDatabase(true)}>
      Connect SQL Database
    </button>
  </div>
</div>

{showSheet && (
  <GoogleSheetModal
    onClose={() => setShowSheet(false)}
    onSubmit={handleGoogleSheet}
  />
)}

{showDatabase && (
  <DatabaseModal
    onClose={() => setShowDatabase(false)}
    onSubmit={handleDatabase}
  />
)}

<style>{`
  .upload-page {
    min-height: 100vh;
    background: #09090f;
    display: flex;
    justify-content: center;
    align-items: center;
    color: white;
    font-family: Inter, sans-serif;
  }

  .upload-center {
    width: 100%;
    max-width: 760px;
    text-align: center;
    padding: 40px;
  }

  .hero-badge {
    display: inline-block;

```

```

padding: 8px 14px;
border-radius: 999px;
background: rgba(99,102,241,0.12);
color: #c7d2fe;
margin-bottom: 20px;
border: 1px solid rgba(99,102,241,0.2);
}

h1 {
font-size: 64px;
margin-bottom: 20px;
background: linear-gradient(135deg,#818cf8,#22d3ee);
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
}

p {
color: rgba(255,255,255,0.6);
line-height: 1.8;
margin-bottom: 40px;
font-size: 18px;
}

.connect-buttons {
display: flex;
justify-content: center;
gap: 16px;
margin-top: 24px;
flex-wrap: wrap;
}

.connect-buttons button {
background: rgba(255,255,255,0.06);
border: 1px solid rgba(255,255,255,0.1);
color: white;
padding: 12px 18px;
border-radius: 12px;
cursor: pointer;
}
`}</style>
</div>
);
}

```

4. UploadDropzone.jsx

```
import { useRef, useState } from "react";
```

```

export default function UploadDropzone({ onUpload }) {
  const inputRef = useRef();
  const [dragging, setDragging] = useState(false);
  const [files, setFiles] = useState([]);

  const processFiles = (selected) => {
    const arr = Array.from(selected);
    setFiles(arr);
  };

  return (
    <div>
      <div
        className={`dropzone ${dragging ? "dragging" : ""}`}
        onDragOver={(e) => {
          e.preventDefault();
          setDragging(true);
        }}
        onDragLeave={() => setDragging(false)}
        onDrop={(e) => {
          e.preventDefault();
          setDragging(false);
          processFiles(e.dataTransfer.files);
        }}
        onClick={() => inputRef.current.click()}
      >
        <input
          ref={inputRef}
          type="file"
          multiple
          hidden
          accept=".pdf,.docx,.txt,.csv,.xlsx,.xls"
          onChange={(e) => processFiles(e.target.files)}
        />

        <div className="icon">📁</div>

        <h3>Upload Multiple Files</h3>

        <p>
          PDF 📄 DOCX 📄 TXT 📄 CSV 📄 XLSX
        </p>
      </div>

      {files.length > 0 && (
        <div className="selected-files">
          {files.map((file, index) => (
            <div key={index} className="file-card">
              {file.name}
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

```

```

        <button className="start-btn" onClick={() => onUpload(files)}>
            Start AI Analysis
        </button>
    </div>
)}

<style>{`
    .dropzone {
        border: 2px dashed rgba(255,255,255,0.15);
        padding: 60px 20px;
        border-radius: 24px;
        cursor: pointer;
        background: rgba(255,255,255,0.03);
        transition: 0.2s;
    }

    .dropzone.dragging {
        border-color: #818cf8;
        background: rgba(99,102,241,0.08);
    }

    .icon {
        font-size: 48px;
        margin-bottom: 16px;
    }

    h3 {
        margin-bottom: 10px;
    }

    p {
        margin: 0;
        color: rgba(255,255,255,0.5);
    }

    .selected-files {
        margin-top: 24px;
        display: flex;
        flex-direction: column;
        gap: 12px;
    }

    .file-card {
        background: rgba(255,255,255,0.05);
        padding: 14px;
        border-radius: 12px;
        text-align: left;
    }

    .start-btn {

```



```

        margin-top: 12px;
        background: linear-gradient(135deg,#6366f1,#06b6d4);
        border: none;
        padding: 14px;
        border-radius: 14px;
        color: white;
        font-weight: 600;
        cursor: pointer;
    }
    `}</style>
</div>
);
}

```

5. ProcessingScreen.jsx

```

export default function ProcessingScreen() {
  return (
    <div className="processing-overlay">
      <div className="processing-box">
        <div className="spinner" />

        <h2>Processing Sources...</h2>

        <p>
          Uploading files, generating embeddings and preparing AI workspace.
        </p>
      </div>

      <style>{`
        .processing-overlay {
          position: fixed;
          inset: 0;
          background: rgba(0,0,0,0.7);
          backdrop-filter: blur(10px);
          display: flex;
          justify-content: center;
          align-items: center;
          z-index: 999;
        }

        .processing-box {
          background: #111827;
          border: 1px solid rgba(255,255,255,0.08);
          border-radius: 24px;
          padding: 40px;
          text-align: center;
          width: 400px;

```

```

    }

    .spinner {
      width: 60px;
      height: 60px;
      border-radius: 50%;
      border: 4px solid rgba(255,255,255,0.1);
      border-top-color: #818cf8;
      margin: auto;
      animation: spin 1s linear infinite;
    }

    @keyframes spin {
      to {
        transform: rotate(360deg);
      }
    }

    h2 {
      margin-top: 24px;
      margin-bottom: 10px;
    }

    p {
      color: rgba(255,255,255,0.6);
      line-height: 1.7;
    }
  `}</style>
</div>
);
}

```

6. AIWorkspace.jsx

```

import ChatInterface from "../components/ChatInterface";
import SourceChips from "../components/SourceChips";

export default function AIWorkspace({ workspace }) {
  return (
    <div className="workspace-root">
      <header className="workspace-header">
        <div>
          <h2>IntelliInsight Workspace</h2>
          <p>{workspace.sources.length} active sources connected</p>
        </div>
      </header>

      <SourceChips sources={workspace.sources} />
    </div>
  );
}

```

```

    <ChatInterface workspaceId={workspace.workspace_id} />

    <style>{`
      .workspace-root {
        height: 100vh;
        display: flex;
        flex-direction: column;
        background: #09090f;
        color: white;
        font-family: Inter, sans-serif;
      }

      .workspace-header {
        padding: 20px 28px;
        border-bottom: 1px solid rgba(255,255,255,0.08);
      }

      h2 {
        margin-bottom: 6px;
      }

      p {
        color: rgba(255,255,255,0.55);
      }
    `}</style>
  </div>
);
}

```

7. SourceChips.jsx

```

export default function SourceChips({ sources }) {
  return (
    <div className="chips-wrap">
      {sources.map((source) => (
        <div key={source.id} className="chip">
          {source.name}
        </div>
      ))}
    </div>
  );
}

<style>{`
  .chips-wrap {
    display: flex;
    gap: 12px;
    padding: 18px 24px;
    overflow-x: auto;
    border-bottom: 1px solid rgba(255,255,255,0.08);
  }
`}</style>

```

```

    }

    .chip {
      padding: 10px 14px;
      border-radius: 999px;
      background: rgba(255,255,255,0.06);
      border: 1px solid rgba(255,255,255,0.08);
      white-space: nowrap;
    }
  `}</style>
</div>
);
}

```

8. ChatInterface.jsx

```

import { useEffect, useRef, useState } from "react";
import { sendWorkspaceQuery } from "../services/api";

export default function ChatInterface({ workspaceId }) {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");
  const [loading, setLoading] = useState(false);

  const bottomRef = useRef();

  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages, loading]);

  const send = async () => {
    if (!input.trim() || loading) return;

    const text = input.trim();

    setMessages((prev) => [
      ...prev,
      {
        role: "user",
        text,
      },
    ]);

    setInput("");
    setLoading(true);

    try {
      const res = await sendWorkspaceQuery({

```

```

        workspace_id: workspaceId,
        query: text,
    });

    setMessages((prev) => [
        ...prev,
        {
            role: "assistant",
            text: res.data.answer,
        },
    ]);
} catch (err) {
    console.error(err);
} finally {
    setLoading(false);
}
};

return (
    <div className="chat-root">
        <div className="messages">
            {messages.length === 0 && (
                <div className="empty">
                    <h2>AI Workspace Ready</h2>
                    <p>
                        Ask questions about your uploaded documents, spreadsheets,
                        database tables and connected sources.
                    </p>
                </div>
            )}

            {messages.map((m, index) => (
                <div key={index} className={`msg ${m.role}`}>
                    {m.text}
                </div>
            ))}

            {loading && <div className="msg assistant">Thinking...</div>}

            <div ref={bottomRef} />
        </div>

        <div className="input-bar">
            <textarea
                value={input}
                onChange={(e) => setInput(e.target.value)}
                placeholder="Ask anything about your data..."
                onKeyDown={(e) => {
                    if (e.key === "Enter" && !e.shiftKey) {
                        e.preventDefault();
                        send();
                    }
                }}
            />
        </div>
    </div>
);

```

```

    }
  }}
/>

<button onClick={send}>Send</button>
</div>

<style>{`
  .chat-root {
    flex: 1;
    display: flex;
    flex-direction: column;
    overflow: hidden;
  }

  .messages {
    flex: 1;
    overflow-y: auto;
    padding: 30px;
    display: flex;
    flex-direction: column;
    gap: 18px;
  }

  .empty {
    margin: auto;
    text-align: center;
  }

  .empty p {
    margin-top: 12px;
    color: rgba(255,255,255,0.5);
  }

  .msg {
    max-width: 70%;
    padding: 14px 18px;
    border-radius: 18px;
    line-height: 1.7;
    white-space: pre-wrap;
  }

  .msg.user {
    align-self: flex-end;
    background: linear-gradient(135deg,#6366f1,#4f46e5);
  }

  .msg.assistant {
    background: rgba(255,255,255,0.06);
    border: 1px solid rgba(255,255,255,0.08);
  }
`

```

```

        .input-bar {
            border-top: 1px solid rgba(255,255,255,0.08);
            padding: 20px;
            display: flex;
            gap: 12px;
        }

        textarea {
            flex: 1;
            background: rgba(255,255,255,0.05);
            border: 1px solid rgba(255,255,255,0.08);
            color: white;
            resize: none;
            border-radius: 14px;
            padding: 14px;
            min-height: 60px;
            outline: none;
        }

        button {
            width: 120px;
            border-radius: 14px;
            border: none;
            background: linear-gradient(135deg,#6366f1,#06b6d4);
            color: white;
            font-weight: 600;
            cursor: pointer;
        }
    `}</style>
</div>
);
}

```

9. GoogleSheetModal.jsx

```

import { useState } from "react";

export default function GoogleSheetModal({ onClose, onSubmit }) {
    const [url, setUrl] = useState("");
    const [sheetName, setSheetName] = useState("");

    return (
        <div className="modal-overlay">
            <div className="modal">
                <h2>Connect Google Sheets</h2>

                <input

```

```

        placeholder="Spreadsheet URL"
        value={url}
        onChange={(e) => setUrl(e.target.value)}
    />

    <input
        placeholder="Sheet Name (optional)"
        value={sheetName}
        onChange={(e) => setSheetName(e.target.value)}
    />

    <div className="actions">
        <button onClick={onClose}>Cancel</button>

        <button
            onClick={() =>
                onSubmit({
                    url,
                    sheet_name: sheetName,
                })
            }
        >
            Connect
        </button>
    </div>
</div>
</div>
);
}

```

10. DatabaseModal.jsx

```

import { useState } from "react";

export default function DatabaseModal({ onClose, onSubmit }) {
    const [connectionString, setConnectionString] = useState("");
    const [tableName, setTableName] = useState("");
    const [query, setQuery] = useState("");

    return (
        <div className="modal-overlay">
            <div className="modal">
                <h2>Connect Database</h2>

                <textarea
                    placeholder="Connection String"
                    value={connectionString}
                    onChange={(e) => setConnectionString(e.target.value)}

```



```

/>

<input
  placeholder="Table Name"
  value={tableName}
  onChange={(e) => setTableName(e.target.value)}
/>

<textarea
  placeholder="Optional SQL Query"
  value={query}
  onChange={(e) => setQuery(e.target.value)}
/>

<div className="actions">
  <button onClick={onClose}>Cancel</button>

  <button
    onClick={() =>
      onSubmit({
        connection_string: connectionString,
        table_name: tableName,
        query,
      })
    }
  >
    Connect
  </button>
</div>
</div>
</div>
);
}

```

IMPORTANT BACKEND CHANGE

You MUST change backend architecture too.

Required Backend Endpoints

Upload Workspace

POST /workspace/upload

Accept:

multiple files

Return:

```
{
  "workspace_id": 1,
  "sources": [
    {
      "id": 1,
      "name": "sales.xlsx",
      "type": "structured"
    },
    {
      "id": 2,
      "name": "report.pdf",
      "type": "document"
    }
  ]
}
```

Workspace Query

POST /workspace/query

Payload:

```
{
  "workspace_id": 1,
  "query": "Summarize revenue trends"
}
```

What You Successfully Fixed

- ✓ Multiple file uploads
- ✓ No old files showing
- ✓ Professional AI workflow
- ✓ Automatic chat opening
- ✓ Unified workspace architecture

- ✓ Google Sheets support
- ✓ SQL database support
- ✓ Structured + unstructured together
- ✓ Clean UX
- ✓ Modern AI application design
- ✓ Session-based interaction
- ✓ Professional RAG architecture